Express Car App - Viva Exam Information

Prepared from the current codebase for exam use. This document explains what the app does, which files are important, which packages and APIs are used, and how image and location features work.

1. App Overview

- Express Car is a Flutter car-rental application with Firebase-based authentication, Firestore storage, image upload, and live location tracking.
- The app starts from **main.dart**, initializes Firebase, loads fallback car data once, shows a splash screen, and then redirects to **AuthWrapper**.
- Role-based routing decides whether the user sees the admin panel, profile completion screen, or the normal home page.
- The main user flow is: sign up or sign in, complete profile if needed, browse cars, book a car, and view booking details and tracking.

2. Authentication and Routing

- **main.dart** initializes Firebase using **firebase_options.dart** and routes to the splash wrapper.
- **AuthWrapper** listens to **FirebaseAuth.authStateChanges()**. If a user is logged in, it checks role and profile completeness from Firestore.
- If the role is **admin**, the app opens **AdminPanelPage**. If the profile is incomplete, it opens **CompleteProfilePage**. Otherwise it opens **HomePage**.
- **signup_page.dart** creates a user with email/password or Google Sign-In, creates the user document in Firestore, and then sends the user to sign-in again.
- **signin_page_new.dart** signs in users and then sends them to **AuthWrapper** so the app can apply the same role/profile checks everywhere.

3. Image Handling Flow

- Image selection happens in **Add_Car.dart** using **image_picker**. The admin can choose either a gallery image or a preset asset image.
- The selected image is uploaded by **imgbb_upload_service.dart**. The app tries ImgBB first through a multipart HTTP request to **https://api.imgbb.com/1/upload**.
- If ImgBB fails, the code falls back to **Firebase Storage** using **firebase_storage**. This prevents the car save process from failing when ImgBB is unavailable.
- The image URL is stored in Firestore inside the **image_url** field for the car document.
- **car_image_widget.dart** contains **normalizeApiImageUrl()** and **CarApiImage**. It cleans malformed URLs, loads network images, shows a loading spinner, and falls back to a local asset image if the URL is invalid or loading fails.
- **car_model.dart** parses image fields from Firestore and also provides fallback asset image paths from **assets/images**.
- **home_page.dart**, **Booked_Car.dart**, **Book_car.dart**, **Favorite.dart**, and **Manage_Cars.dart** all use **CarApiImage** or normalized network images to display the car image safely.

4. Location Tracking Flow

- **Add_Car.dart** also tries to capture the car owner's current GPS location when the car is saved. This is optional; if location is unavailable, the car still saves.
- **car_location_tracking_service.dart** is the main tracking service. It uses **Geolocator** to get permission, start a position stream, and update owned cars in Firestore.
- The service updates both **cars** and **fleet_items** collections using a Firestore batch so multiple owned vehicles can receive the same live location.
- Each update stores **current_location** as a **GeoPoint** plus **location_lat**, **location_lng**, **location_accuracy_m**, **location_speed_mps**, **location_heading_deg**, and **location_updated_at**.

- **HomePage** starts this tracking service for the current owner, so location updates continue in the background while the app is open.
- **live_booking_tracking_page.dart** uses **flutter_map** and **latlong2** to show the car marker and the user marker on a map. It reads the latest Firestore location fields and also keeps the user position updated live.

5. External Packages and APIs Used

- **firebase_core** - Firebase app initialization
- **firebase_auth** - Email/password and Google authentication
- **cloud_firestore** - Car, booking, and user data storage
- **firebase_storage** - Fallback storage for car images
- **google_sign_in** - Google account sign-in flow
- **image_picker** - Picking images from gallery
- **http** - Uploading images to ImgBB
- **geolocator** - Getting GPS location and permission handling
- **flutter_map** - Showing the live tracking map
- **latlong2** - Latitude and longitude point objects
- **cloudinary_public** - Used in admin fleet/driver document upload flow
- **pinput** - OTP/PIN input UI
- **multi_select_flutter** - Selecting multiple car features

6. Important Firestore Collections

- **users** - user profile, role, favorites, presence information, and profile completion fields.
- **cars** - main car listings, image URL, owner info, pricing, type, and live location fields.
- **fleet_items** - alternate fleet collection that also receives live location updates.
- **bookings** - booking records, dates, pricing, pickup location, and driverless document details.
- **counters** - stores the car ID counter so new cars get a unique numeric ID.

7. Viva-Friendly File List

- **main.dart** - app entry point, Firebase setup, splash to auth routing.
- **auth_wrapper.dart** - role-based routing and profile check.
- **signup_page.dart** - register user and create Firestore user document.
- **signin_page_new.dart** - login and redirect to AuthWrapper.
- **Add_Car.dart** - add a car, upload image, capture location, save to Firestore.
- **imgbb_upload_service.dart** - ImgBB upload helper with error handling and API key support.
- **car_image_widget.dart** - safe image display and fallback asset support.
- **car_location_tracking_service.dart** - live owner car location updates to Firestore.
- **car_model.dart** - parse car data from Firestore into model objects.
- **home_page.dart** - user car browsing page and tracking start point.
- **Book_car.dart** - booking form and booking save flow.
- **booking_details_page.dart** - booking summary and tracking button.
- **live_booking_tracking_page.dart** - map view showing live car and user positions.

8. Short Viva Answer You Can Say

"This app is a Flutter-based car rental system. Firebase Auth is used for login, Firestore stores users, cars, and bookings, ImgBB and Firebase Storage handle image uploads, and Geolocator plus Flutter Map handle live location tracking. The AuthWrapper controls role-based routing, and the Add Car and Live Tracking files are the main files for image and location features."

Prepared from the current workspace codebase on 20 April 2026.